BotXRGame — Session 3 Progress Log

Controller Teleoperation Demo: Thumbstick → Twist → Spaceship + ROS Publishing

Date: July 2026 | Author: swapnil-0

1. Session Goal & Outcome

Goal: Build a self-contained demo where the physical Samsung Galaxy XR controller drives a virtual spaceship (standing in for the robot), converts thumbstick input into a geometry_msgs/Twist message, publishes it to the ROS /cmd_vel topic, and displays live values on an in-headset HUD. The app must work standalone with no PC dependency at demo time.

Outcome: Achieved. The controller thumbstick moves the spaceship in simulation and publishes Twist messages to /cmd_vel. A runtime IP entry screen lets the user point the app at any ROS host without rebuilding. The app runs fully on-device.

2. Demo Architecture

Three C# scripts drive the demo:

Script	Attached To	Responsibility
RobotController.cs	Fighter03 (spaceship)	Reads thumbstick, moves spaceship, formats + publishes Twist to /cmd_
CmdVelHUD.cs	Canvas	Displays live linear.x, angular.z, topic, and connection status
ROSIPConfig.cs	Canvas	Runtime IP entry screen; switches between IP input panel and HUD pane

3. ROS Interface

Parameter	Value	Notes
Topic	/cmd_vel	Standard mobile robot velocity command topic
Message Type	geometry_msgs/Twist	linear.x = forward/back, angular.z = turn
Publish Rate	10 Hz	Configurable in RobotController Inspector
ROS Port	10000 (FIXED)	Not user-editable at runtime - see note below
ROS IP	User-entered at runtime	Editable via in-headset IP entry screen

IMPORTANT - PORT IS FIXED: The ROS connection port is hardcoded to 10000, the default ros_tcp_endpoint port. Only the IP address can be changed at runtime through the in-headset entry screen. If the ROS Dev runs ros_tcp_endpoint on a non-default port, the app must be rebuilt in Unity (or a port input field added). For the current demo, the ROS Dev must use the default port 10000.

4. Runtime IP Configuration (No PC Needed)

To avoid any dependency on a PC or the ROS Dev at demo time, the app presents an IP entry screen at startup:

1. App launches showing the IP Input Panel with a text field and two buttons (Connect, Skip).
2. User enters the ROS host IP. The last-used IP is remembered via PlayerPrefs and pre-filled.
3. Connect: applies the IP to the ROS connection and switches to the HUD panel.
4. Skip: enters simulation-only mode (spaceship still moves, no ROS publishing needed).

Port stays fixed at 10000 in both paths.

---

5. Key Problem Solved: Controller Input Reading Zero

Symptom: The controller ray tracked position/rotation correctly (ray moved with the controller, hover detection worked), but thumbstick and trigger values always read zero — regardless of how the input was read (XRI action references, direct InputDevices API, and inline InputSystem bindings were all tried).

Root cause: The enabled OpenXR interaction profile was the Khronos Simple Controller Profile, which only specifies pose tracking plus select/menu buttons. It has NO thumbstick or trigger axis in its specification, so the OpenXR runtime provided no data for those inputs.

Fix: In Edit > Project Settings > XR Plug-in Management > OpenXR > Android tab > Interaction Profiles, removed the Khronos Simple Controller Profile and added the Oculus Touch Controller Profile. The Android XR runtime maps the Samsung Galaxy XR controller to the Oculus Touch layout, which exposes a full thumbstick, trigger, and grip. After this change, thumbstick and trigger values were read correctly and the spaceship moved.

Final input binding (InputActionReference approach):

RobotController Field	Bound Action
Move Action	XRI Right/Thumbstick
Trigger Action	XRI Right Interaction/Activate Value

---

6. Other Fixes This Session

6.1 UI Ray Interaction

Added a Ray Interactor under XR Origin > Camera Offset. Added a Tracked Pose Driver (Input System) component bound to XR Right/Position and XR Right/Rotation so the ray follows the physical controller. Added an XR Interaction Manager to the scene. Replaced the Canvas's Graphic Raycaster with a Tracked Device Graphic Raycaster so the XR ray can click UI. Set the Ray Interactor UI Press Input to XRI Right Interaction/Activate.

6.2 UI Layout

Fixed overlapping text and buttons by spacing HUD text vertically (StatusText, LinearXText, AngularZText, TopicText) and stacking the IP panel elements (title, input field, Connect, Skip, status). Sized the World Space Canvas to scale 0.001 with appropriate width/height so it reads as a comfortable floating panel.

6.3 Panel Flow Bug

Corrected ROSIPConfig so the app starts on the IP Input Panel (not the HUD), and Connect/Skip correctly switch to the HUD panel. An earlier debug build had these reversed.

6.4 Connection Status Check

Replaced the placeholder CheckConnection method (which always reported Connected) with a real check against the ROS connection error state, so the HUD accurately shows Waiting for ROS vs Connected.

---

7. Current Status

✓ Samsung XR controller thumbstick drives spaceship in simulation
✓ Thumbstick input converted to geometry_msgs/Twist
✓ Twist published to /cmd_vel at 10 Hz
✓ Oculus Touch Controller Profile enabled (fixes zero-input issue)
✓ Ray Interactor follows physical controller via Tracked Pose Driver
✓ XR ray can click UI buttons (Tracked Device Graphic Raycaster)
✓ Runtime IP entry screen - no PC needed at demo time
✓ Last-used IP remembered via PlayerPrefs
✓ Skip button for simulation-only mode
✓ Live HUD shows linear.x, angular.z, topic, connection status
✓ Real ROS connection-state check on HUD
✓ App builds and runs standalone on Samsung Galaxy XR
✓ All work pushed to GitHub

Known Limitations / Next Steps

■ ROS port is FIXED at 10000 - non-default ports require a rebuild or a new port input field
■ IP must be typed with the controller - consider preset IP buttons for faster demo setup
■ Not yet tested against a live ROS 2 Humble endpoint (pending RubikPi from ROS Dev)
■ Twist coordinate conventions to be confirmed with ROS Dev against real robot
■ Remove any remaining debug status strings before final production build

---